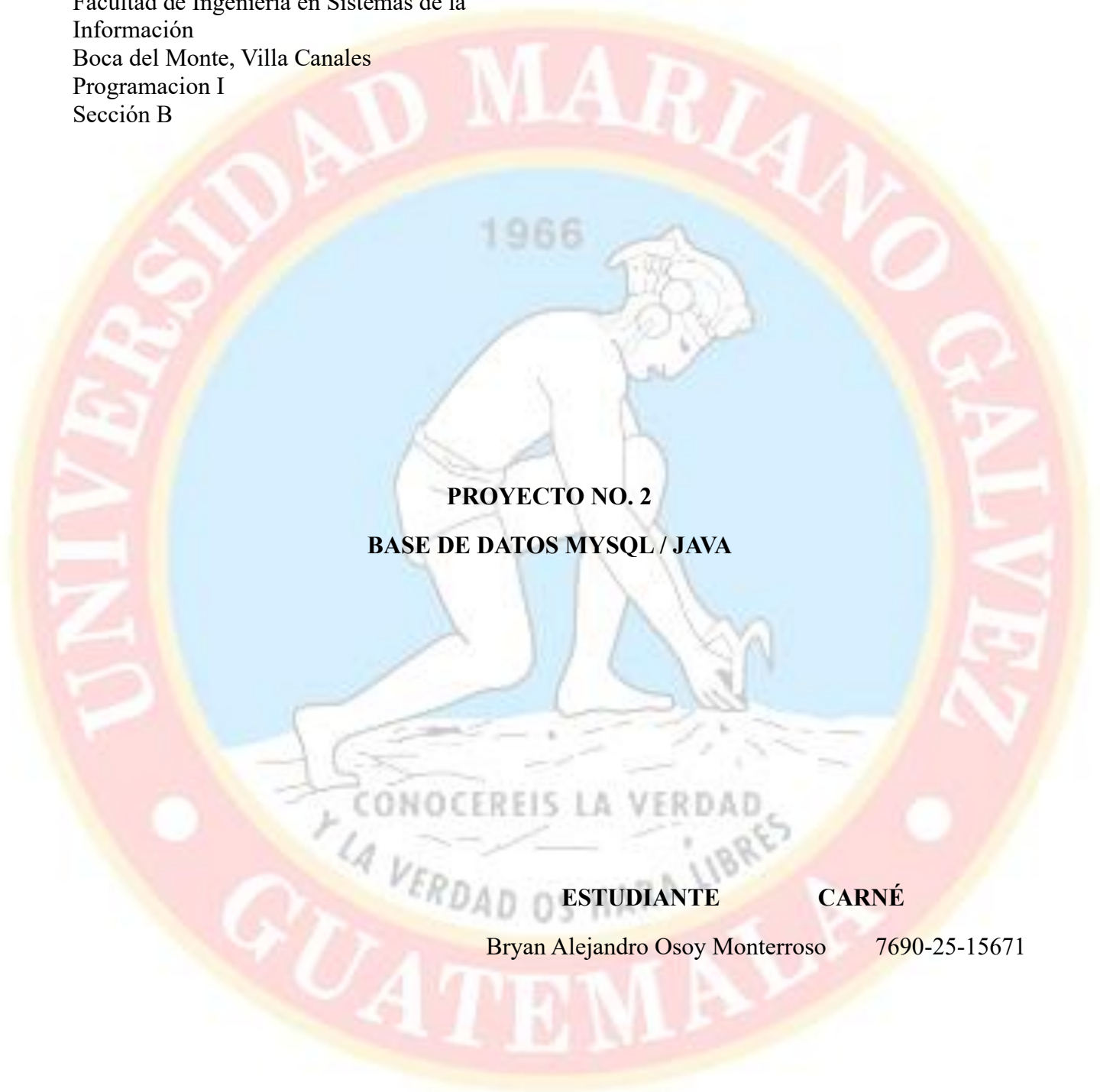


Universidad Mariano Gálvez de Guatemala
Facultad de Ingeniería en Sistemas de la
Información
Boca del Monte, Villa Canales
Programación I
Sección B



PROYECTO NO. 2
BASE DE DATOS MYSQL / JAVA

ESTUDIANTE	CARNÉ
Bryan Alejandro Osoy Monterroso	7690-25-15671

ÍNDICE

INTRODUCCION.....	3
ENLACES.....	3
JUSTIFICACION	4
OBJETIVOS.....	5
Objetivo General.....	5
Objetivos Específicos	5
ANALISIS Y DESARROLLO	6
Clases, Objetos y Paquetes	6
Relación entre clases.....	6
descripción de base de datos.....	7
Tabla: alumnos	7
Tabla: notas	7
Descripción de Menús y Opciones.....	8
Librerías, Clases y Métodos Principales.....	9
Métodos importantes implementados.....	9
MANUAL DE USUARIO.....	10
Requisitos del Sistema	10
Configuración Inicial.....	10
Guía de Uso por Opción	11
Opción 1 – Ingreso de Alumnos.....	11
Opción 2 – Ingreso de Notas.....	11
Opción 3 – Eliminar Alumnos	11
Opción 4 – Actualizar Datos.....	11
Opción 5 – Buscar Alumnos.....	12
Opción 6 – Obtener Promedios.....	12
Opción 7 – Listar Alumnos	12
Opción 8 – Salir.....	12
CONCLUSION.....	13
REFERENCIAS BIBLIOGRAFICAS.....	14

INTRODUCCION

El presente proyecto consiste en el desarrollo de una aplicación de consola en Java que permite gestionar el control de notas de los alumnos del curso de Programación I, específicamente para las secciones A y B. La aplicación se conecta a una base de datos relacional MySQL denominada SistemaNotas, donde se almacena de forma persistente toda la información académica.

El sistema permite al catedrático realizar las operaciones fundamentales de gestión: registrar nuevos alumnos, ingresar y actualizar calificaciones, eliminar registros, buscar alumnos y obtener promedios por sección. La arquitectura del proyecto aplica el patrón de diseño DAO (Data Access Object), separando la lógica de presentación de la lógica de acceso a datos, lo que favorece al mantenimiento y escalabilidad del código.

El proyecto integra los conocimientos del curso sobre programación orientada a objetos, conexión a bases de datos desde Java mediante JDBC, y el uso del driver MySQL Connector/J.

ENLACES

En este apartado se comparten los enlaces de los repositorios y haci como el video publicado en VIDEO PUBLICADO EN YOUTUBE:

<https://youtu.be/QCw1EZcWQL8>

REPOSITORIO GIT HUB:

<https://github.com/bosoym-coder/PROYECTO-2---CONTROL-DE-NOTAS>

JUSTIFICACION

La gestión manual de calificaciones a través de hojas sueltas o herramientas no especializadas implica riesgos de pérdida de información, errores de cálculo y dificultad para consultar datos de forma rápida. Un sistema automatizado elimina estos problemas al centralizar la información en una base de datos con integridad referencial.

La elección de Java como lenguaje de implementación obedece a su solidez en aplicaciones empresariales, su compatibilidad con JDBC para la conexión a bases de datos relacionales y al marco del curso de Programación I. El uso de MySQL como gestor de base de datos se justifica por su amplia adopción, rendimiento y compatibilidad con el conector oficial de Java.

Adicionalmente, aplicar el patrón DAO permite que el código sea más organizado y fácil de mantener, ya que cualquier cambio en la capa de datos no afecta la lógica de presentación. El patrón Singleton en la clase Conexion garantiza que exista una sola conexión activa durante toda la ejecución de la aplicación, optimizando el uso de recursos.

OBJETIVOS

Objetivo General

Desarrollar un sistema de control de notas de alumnos en lenguaje Java con conexión a una base de datos MySQL, que permita al catedrático gestionar la información académica de las secciones A y B mediante un menú de consola.

Objetivos Específicos

- Diseñar y crear una base de datos relacional (SistemaNotas) con las tablas necesarias para almacenar alumnos y calificaciones.
- Implementar las operaciones CRUD sobre los registros de alumnos y notas mediante la capa de acceso a datos DAO.
- Aplicar el patrón Singleton para gestionar de forma eficiente la conexión a la base de datos.
- Calcular automáticamente la nota final de cada alumno como la suma de zona (0–60) y examen (0–40).
- Permitir búsqueda de alumnos por carnet o por nombre/apellido con resultados paginados en consola.
- Implementar el listado de alumnos por sección con opciones de ordenamiento por carnet, nombre, apellido o nota.
- Obtener y mostrar el promedio de notas por sección utilizando funciones de agregación SQL.

ANALISIS Y DESARROLLO

Clases, Objetos y Paquetes

Toda la aplicación está organizada dentro de carpeta **bdnotas**. La estructura del proyecto en Visual Studio Code muestra las clases compiladas en el directorio **bin**, **bdnotas** y el driver de MySQL (mysql-connector-j-8.4.0.jar) en la carpeta lib. A continuación, se describe cada clase:

Clase	Tipo	Actividad
Alumno	Modelo (POJO)	Representa la entidad alumno. Contiene atributos carnet, nombres, apellidos
Nota	Modelo (POJO)	Representa la entidad nota. Contiene id, carnet, zona, examen y nota Final. La nota final se recalcula automáticamente en los setters setZona() y setExamen().
AlumnoDAO	Acceso a Datos	Gestiona todas las operaciones CRUD sobre la tabla alumnos: insertar, existe Carnet, eliminar, actualizar, buscar Por Carnet, buscar Por Nombre, listar Por sección Ordenado y obtener Promedio.
NotaDAO	Acceso a Datos	Gestiona las operaciones CRUD sobre la tabla notas: insertar Nota, tiene Nota, obtener Nota y actualizar Nota.
conexión	Infraestructura	Implementa el patrón Singleton. Mantiene una única instancia de java.sql.Connection para toda la ejecución. Establece getConnection() y cerrarConexion().
Main	Controlador/Vista	Clase principal. Contiene el ciclo do-while del menú, los métodos de cada opción (ingresar Alumno, ingresar Notas, etc.) y los métodos utilitarios leer Entero() y leer Double() con validación de entrada.

Relación entre clases

La clase Main crea instancias de AlumnoDAO y NotaDAO al inicio de la ejecución. Ambas clases DAO obtienen la conexión invocando Conexion.getConnection(). AlumnoDAO recibe y devuelve objetos de tipo Alumno; NotaDAO recibe y devuelve objetos de tipo Nota. Al finalizar el programa (opción 8), Main invoca Conexion.cerrarConexion() para liberar el recurso.

descripción de base de datos

La base de datos se denomina SistemaNotas y se ubica en el servidor MySQL local (localhost:3306). Fue creada mediante un script SQL que incluye sentencias DDL (CREATE TABLE) y datos de prueba (INSERT INTO). Al momento de ejecutar MySQL Workbench indica la creación correcta de ambas tablas con sus datos de prueba.

Tabla: alumnos

Campo	Tipo de Dato	Restricción	Descripción
carnet	VARCHAR(15)	PK, NOT NULL	Identificador único del alumno.
nombres	VARCHAR(60)	NOT NULL	Nombres del alumno.
apellidos	VARCHAR(60)	NOT NULL	Apellidos del alumno.
seccion	CHAR(1)	NOT NULL	Sección asignada: A o B.

Tabla: notas

Campo	Tipo de Dato	Restricción	Descripción
id	INT	PK, AUTO_INCREMENT	Identificador autonumérico.
carnet	VARCHAR(15)	FK → alumnos(carnet)	Llave foránea que referencia la tabla alumnos.
zona	DECIMAL(5,2)	DEFAULT 0	Puntos de zona (rango 0–60).
examen	DECIMAL(5,2)	DEFAULT 0	Puntos de examen (rango 0–40).
nota_final	DECIMAL(5,2)	DEFAULT 0	Suma de zona + examen (0–100).

La relación entre ambas tablas se establece mediante una llave foránea con las cláusulas ON DELETE CASCADE y ON UPDATE CASCADE. Esto garantiza que al eliminar un alumno, sus notas sean eliminadas automáticamente, manteniendo la referencial de la base de datos.

Descripción de Menús y Opciones

El programa presenta en consola un menú principal con 8 opciones que permite al catedrático seleccionar la acción deseada mediante entrada numérica. A continuación, se describe en detalle el funcionamiento de cada opción:

#	Opción	Descripción del Funcionamiento
1	Ingreso de Alumnos	Solicita al usuario el carné, nombres, apellidos y sección (A o B). Antes de insertar verifica con <code>existeCarnet()</code> que el carnet no esté duplicado. La sección se valida con un bucle hasta que el usuario ingrese A o B.
2	Ingreso de Notas	Solicita el carnet del alumno y lo busca en la BD. Valida el rango de zona (0–60) y examen (0–40) con bucles. Si el alumno ya tiene notas registradas, pregunta si desea sobrescribir. Luego inserta o actualiza según corresponda.
3	Eliminar Alumnos	Permite buscar al alumno por carnet o por nombre/apellido. En búsqueda por nombre muestra una lista numerada para seleccionar. Antes de eliminar solicita confirmación S/N. La eliminación en la tabla alumnos aplica CASCADE a la tabla notas.
4	Actualizar Datos	Busca al alumno por carnet y muestra sus datos actuales. Ofrece un submenú con 5 opciones: actualizar nombres, apellidos, sección, notas, o todos a la vez. Cada opción valida el nuevo valor antes de persistirlo.
5	Buscar Alumnos	Permite buscar por carnet (retorna un único resultado) o por nombre/apellido (retorna lista). Para cada resultado muestra los datos del alumno y su nota registrada o el mensaje 'Sin notas registradas'.
6	Obtener Promedios	Calcula y muestra el promedio de <code>nota_final</code> de la Sección A y de la Sección B mediante una consulta SQL con <code>AVG()</code> e <code>INNER JOIN</code> entre las tablas alumnos y notas.
7	Listar Alumnos	Solicita la sección (A o B) y el criterio de ordenamiento (carnet, nombre, apellido o nota). Muestra una tabla formateada con carnet, nombres, apellidos y nota final. Al final indica el total de alumnos listados. Ordenar por nota usa <code>COALESCE</code> para manejar alumnos sin nota.
8	Salir	Cierra la conexión a la base de datos llamando <code>Conexion.cerrarConexion()</code> y termina el programa mostrando un mensaje de despedida.

Librerías, Clases y Métodos Principales

A continuación se listan las principales librerías y clases de Java utilizadas en el proyecto:

Librería / Clase	Paquete	Uso en el Proyecto
Connection	java.sql	Representa la conexión activa a la base de datos MySQL.
DriverManager	java.sql	Crea la conexión mediante getConnection(url, user, pass).
PreparedStatement	java.sql	Ejecuta consultas SQL de forma segura para evitar errores o accesos no autorizados.
ResultSet	java.sql	Recorre los resultados de consultas SELECT.
SQLException	java.sql	Captura errores de acceso a la base de datos.
ArrayList / List	java.util	Almacena listas de objetos Alumno devueltas por las búsquedas.
Scanner	java.util	Lee la entrada del usuario desde la consola (System.in).
mysql-connector-j-8.4.0.jar	com.mysql.cj.jdbc	Driver JDBC oficial de MySQL. Registrado con Class.forName().

Métodos importantes implementados

- leerEntero(String msg): Método utilitario en Main, para garantizar que el usuario ingrese un número entero válido, reintentando en caso de error.
- leerDouble(String msg): Igual al anterior, pero para valores decimales, usado en el ingreso y actualización de notas.
- listarPorSeccionOrdenado(String seccion, String orden): Construye dinámicamente ORDER BY usando un switch, incluyendo COALESCE para notas nulas.
- obtenerPromedio(String seccion): Ejecuta AVG(n.nota_final) con INNER JOIN para obtener el promedio de una sección específica.

MANUAL DE USUARIO

Requisitos del Sistema

- Java Development Kit (JDK) 8 o superior instalado.
- Servidor MySQL activo en localhost, puerto 3306.
- Driver mysql-connector-j-8.4.0.jar incluido en la carpeta lib del proyecto.
- Base de datos SistemaNotas creada ejecutando el script SQL provisto.
- IDE recomendado: Visual Studio Code con extensión Java Extension Pack.

Configuración Inicial

Antes de ejecutar la aplicación es necesario:

1. Ejecutar el script SQL en MySQL Workbench o desde la terminal para crear la base de datos SistemaNotas y sus tablas.
2. Verificar que la clase Conexion.java tenga las credenciales correctas (usuario y contraseña del servidor MySQL local).
3. Asegurarse de que el archivo mysql-connector-j-8.4.0.jar esté referenciado en el classpath del proyecto.
4. Compilar el proyecto y ejecutar la clase Main desde el IDE o desde la terminal con: `java -cp "bin;lib/mysql-connector-j-8.4.0.jar" bdnotas.Main`

Guía de Uso por Opción

Opción 1 – Ingreso de Alumnos

Al seleccionar la opción 1, el sistema solicita el carnet (identificador único), nombres completos, apellidos y la sección (A o B). Si el carnet ya existe en la base de datos, el sistema notifica el error y regresa al menú sin insertar. La sección solo acepta los valores A o B; cualquier otro valor obliga a reingresar el dato.

Opción 2 – Ingreso de Notas

Se solicita el carnet del alumno. Si el alumno existe, se muestran sus datos y se solicita la zona (valor entre 0 y 60) y el examen (valor entre 0 y 40). Si el alumno ya tiene notas registradas, el sistema pregunta si se desean sobrescribir. La nota final se calcula y muestra automáticamente como zona + examen.

Opción 3 – Eliminar Alumnos

Permite buscar al alumno a eliminar ya sea por carnet o por nombre/apellido. En la búsqueda por nombre se puede mostrar más de un resultado; el usuario selecciona el deseado con su número de lista. Antes de confirmar la eliminación, el sistema muestra los datos del alumno y solicita confirmación con S o N. Al eliminar un alumno, sus notas se eliminan automáticamente por la restricción CASCADE.

Opción 4 – Actualizar Datos

Después de localizar al alumno por carnet y visualizar sus datos actuales, se muestra un submenú que permite modificar: (1) solo nombres, (2) solo apellidos, (3) solo sección, (4) solo notas, o (5) todos los datos incluyendo notas. Cada campo actualizado se valida antes de guardarse en la base de datos.

Opción 5 – Buscar Alumnos

Ofrece dos modalidades de búsqueda: por carnet (resultado único) o por nombre/apellido (puede retornar múltiples resultados). Para cada alumno encontrado se muestran sus datos personales y la nota registrada. Si el alumno no tiene notas, se indica con el mensaje 'Sin notas registradas'.

Opción 6 – Obtener Promedios

Muestra en pantalla el promedio de la nota final de todos los alumnos de la Sección A y de la Sección B. El cálculo se realiza directamente en MySQL mediante la función AVG(), considerando únicamente los alumnos que tienen notas registradas.

Opción 7 – Listar Alumnos

Solicita primero la sección (A o B) y luego el criterio de ordenamiento: (1) por carnet, (2) por nombre, (3) por apellido, (4) por nota o (5) sin orden específico. El resultado se presenta en formato de tabla con columnas alineadas que muestran carnet, nombres, apellidos y nota final. Al final se indica el total de alumnos de la sección.

Opción 8 – Salir

Finaliza la ejecución del programa. Antes de salir, el sistema cierra formalmente la conexión a la base de datos mediante `Conexion.cerrarConexion()` y muestra el mensaje 'Cerrando el sistema... ¡Hasta luego!'.

CONCLUSION

El desarrollo de este proyecto permitió aplicar de forma práctica los conocimientos del curso de Programación I, integrando la programación orientada a objetos en Java con el manejo de una base de datos relacional MySQL. La implementación del patrón DAO mantuvo el código organizado y con responsabilidades bien definidas, mientras que el patrón Singleton garantizó un uso eficiente de la conexión a la base de datos.

El proyecto se realizó con todos los requerimientos establecidos, resultando en Programa funcional, con validaciones de entrada y manejo de errores, que demuestra la importancia de planificar correctamente tanto la estructura del código como el diseño de la base de datos antes de comenzar a desarrollar o la construcción de la misma.



REFERENCIAS BIBLIOGRAFICAS

Oracle Corporation. (2024). Java SE Documentation – JDBC API. Oracle. Recuperado de <https://docs.oracle.com/en/java/>

Oracle Corporation. (2024). MySQL 8.0 Reference Manual. Oracle. Recuperado de <https://dev.mysql.com/doc/refman/8.0/en/>

MySQL AB. (2024). MySQL Connector/J 8.4 Developer Guide. Oracle. Recuperado de <https://dev.mysql.com/doc/connector-j/en/>

Deitel, P. & Deitel, H. (2020). Java: How to Program, Early Objects (11th ed.). Pearson Education.

Sierra, K. & Bates, B. (2005). Head First Java (2nd ed.). O'Reilly Media.

Coronel, C., Morris, S. & Rob, P. (2017). Database Systems: Design, Implementation, & Management (13th ed.). Cengage Learning.